

Here is the updated logic for your routes to pull data directly from **Postgres** instead of your temporary in-memory array.

Node.js Lane (Express)

Update the `GET /tasks` and `GET /tasks/:id` endpoints in your `server.js` file to perform parameterized database queries:

javascript

```
const express = require('express');
const pool = require('./db'); // Import the Postgres connection pool
const app = express();
const PORT = 3000;

app.use(express.json());

// GET /tasks - Reads straight from Postgres
app.get('/tasks', async (req, res) => {
  try {
    let sql = 'SELECT * FROM tasks WHERE 1=1';
    const params = [];
    const { done, search } = req.query;

    if (done !== undefined) {
      params.push(done === 'true');
      sql += ` AND done = ${params.length}`;
    }

    if (search !== undefined && search.trim() !== "") {
      params.push(`%${search.toLowerCase()}%`);
      sql += ` AND LOWER(title) LIKE ${params.length}`;
    }

    sql += ' ORDER BY id ASC;';

    const result = await pool.query(sql, params);
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

```
// GET /tasks/:id - Reads a single item securely via placeholders
app.get('/tasks/:id', async (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  if (isNaN(taskId)) {
    return res.status(400).json({ error: "Invalid task ID format" });
  }

  try {
    const result = await pool.query('SELECT * FROM tasks WHERE id = $1;', [taskId]);

    if (result.rows.length === 0) {
      return res.status(404).json({ error: `Task ${taskId} not found` });
    }

    res.json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});

// (Keep remaining placeholder endpoints below until we update them in subsequent steps)
app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```

Use code with caution.

Python Lane (FastAPI)

Update your `main.py` routing logic to fetch rows directly via connection wrappers using structural placeholders:

python

```
from fastapi import FastAPI, HTTPException, status
from typing import Optional, List
from database import get_db_connection

app = FastAPI()
```

```
# GET /tasks - Reads straight from Postgres
@app.get("/tasks")
def read_tasks(done: Optional[bool] = None, search: Optional[str] = None):
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                sql = "SELECT * FROM tasks WHERE 1=1"
                params = []

                if done is not None:
                    sql += " AND done = %s"
                    params.append(done)

                if search is not None and search.strip():
                    sql += " AND LOWER(title) LIKE %s"
                    params.append(f"%{search.lower()}%")

                sql += " ORDER BY id ASC;"

                cur.execute(sql, params)
                return cur.fetchall()
    except Exception as e:
        raise HTTPException(status_code=500, detail={"error": str(e)})

# GET /tasks/{id} - Reads a single item securely via placeholders
@app.get("/tasks/{id}")
def read_task(id: int):
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT * FROM tasks WHERE id = %s;", (id,))
                task = cur.fetchone()

                if task is None:
                    raise HTTPException(
                        status_code=404,
                        detail={"error": f"Task {id} not found"}
                    )
                return task
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail={"error": str(e)})
```

Use code with caution.

Checkpoint Verification

Fire up your server application and hit your test paths using your local machine terminal line:

```
bash
```

```
# Verify fetching all rows
```

```
curl -i http://localhost:3000/tasks
```

```
# Verify failing on an unknown key
```

```
curl -i http://localhost:3000/tasks/999
```